A Survey on Smart Contract Vulnerability Detection Using Machine Learning, Large Language Models, and Multi-Agent Systems

Abel S Mathew, Abraham Santhosh , Diya Pauly, Megha Ajith
Alka Vijay
Department of Computer Engineering
College of Engineering Chengannur
APJ Abdul Kalam Technological University
Kerala, India

***Abstract*—Smart contracts are self-executing programs deployed on blockchain platforms, but vulnerabilities in their code can lead to serious security issues and financial losses. Traditional approaches to smart contract security mainly rely on static analysis, symbolic execution, and predefined vulnerability patterns. In recent years, machine learning, transformer-based models, large language models (LLMs), and multi-agent systems have been explored to make vulnerability detection more effective and intelligent. This survey reviews these recent developments, covering approaches based on static analysis, machine learning, transformer models, generative LLMs, combinations of LLMs with program analysis, and multi-agent collaboration. The reviewed studies include Slither, hybrid attention networks, transformer encoders, ensemble multi-label learning, GPTLENS, SmartLLM, GPTScan, ChatDev, ChatEval, multi-agent debate, LLM-SmartAudit, and prompt-engineered LLM-based detection. These approaches are compared based on their objectives, techniques, datasets, reported results, and limitations. The findings show a clear shift from rule-based and feature-based techniques toward models that can understand code context and support more advanced reasoning. Despite this progress, several challenges remain, including false positives, limited vulnerability coverage, hallucinations in LLM-based systems, the lack of large Solidity-specific datasets, limited explainability, scalability issues, and the need for reliable validation of generated findings. Overall, integrating program analysis with domain-adapted LLMs and structured multi-agent verification appears to be a promising direction for developing more accurate, reliable, and explainable smart contract security auditing systems.**

***Index Terms*—Smart Contracts, Vulnerability Detection, Solidity, Machine Learning, Transformers, Large Language Models, LLMs, Multi-Agent Systems, Static Analysis, Program Analysis.**

I. Introduction

Smart contracts enable blockchain applications to execute predefined logic without requiring a conventional central authority. Since deployed contracts can control digital assets and interact with other contracts, programming errors and security vulnerabilities can have serious consequences. Vulnerabilities such as reentrancy, improper access control, transaction-order dependence, unsafe use of tx.origin, and other logic errors have therefore become important research problems.

Early smart contract auditing approaches mainly relied on static analysis and symbolic reasoning. Slither is a representative static analysis framework that provides detectors and intermediate representations for analysing Solidity contracts [1]. Although such tools are useful for known patterns, they can produce false positives, miss semantic or application-specific logic flaws, and require manually designed analyses.

Machine learning and deep learning introduced data-driven vulnerability detection. Hybrid attention models and transformer encoders have been used to learn code representations and classify vulnerable contracts or code fragments [2], [3]. Ensemble multi-label methods further address the fact that a contract can contain more than one vulnerability category [4]. More recently, LLMs have been investigated for smart contract auditing because they can interpret source code and provide natural-language reasoning. GPTLENS, SmartLLM, and GPTScan demonstrate different ways of using LLMs either directly or together with program-analysis information [5]–[7].

Research in general LLM reasoning has also shown that multiple agents can improve reliability by allowing independent reasoning, debate, evaluation, and revision. Multi-agent debate and ChatEval demonstrate the value of interaction among language-model agents for reasoning and evaluation tasks [8], [9]. ChatDev provides a software-development example in which specialized agents collaborate through structured communication [10]. These ideas have been adapted to smart contract auditing in systems such as LLM-SmartAudit, where multiple agents cooperate to detect and analyse vulnerabilities [12].

This survey reviews these approaches, compares their strengths and limitations, and identifies research gaps toward reliable, explainable, and scalable smart contract vulnerability detection systems.

II. REVIEW OF RECENT SMART CONTRACT VULNERABILITY DETECTION APPROACHES

Feist et al. introduced Slither, a static analysis framework designed specifically for smart contracts [1]. The framework analyses Solidity programs using intermediate representations and a collection of security detectors. It can identify several common vulnerability patterns and provides information useful for developers and auditors. A major advantage of Slither is its speed and its ability to integrate program-analysis information into a development workflow. However, static detectors are strongly dependent on predefined rules and program-analysis heuristics. Therefore, semantic logic vulnerabilities and previously unseen patterns can remain difficult to detect.

Wu et al. proposed a Smart Contract Vulnerability Detection Based on Hybrid Attention Mechanism Model (HAM) [2]. The approach applies attention mechanisms to learn important relationships within smart contract code and improve vulnerability classification. Compared with purely handcrafted features, the model attempts to learn relevant code representations automatically. The study demonstrates the potential of attention-based deep learning for smart contract security. However, the approach remains dependent on the quality and representativeness of its training data and is less suitable for explaining complex vulnerability reasoning to auditors.

Le et al. investigated robust vulnerability detection in Solidity-based Ethereum smart contracts using fine-tuned transformer encoder models [3]. Transformer encoders provide contextual representations of source code and can learn relationships between tokens over longer contexts. Fine-tuning a pretrained encoder for vulnerability detection improves the ability of the model to recognize security-related patterns in Solidity. The study highlights the usefulness of pretrained code representations, but transformer classifiers still depend on labelled datasets and may not naturally provide the detailed reasoning expected from a security auditor.

Alnuman et al. proposed an ensemble multi-label machine learning model for Solidity smart contract vulnerability detection [4]. The multi-label formulation is important because a single contract may exhibit several vulnerability types simultaneously. The ensemble design combines predictive models to improve classification performance, with the reported F1 scores ranging from approximately 82.0% to 99.9% across the evaluated vulnerability datasets. Despite the strong reported results, supervised models require representative labelled data and can struggle when deployed on vulnerability patterns that differ from the training distribution.

Hu et al. presented GPTLENS, described as a new perspective on large language model-powered smart contract vulnerability detection [5]. The study explores the use of LLMs for interpreting smart contract code and detecting security problems. Compared with conventional classifiers, an LLM can use broader code context and generate natural-language explanations. This makes the approach attractive for auditing and vulnerability triage. Nevertheless, direct LLM-based detection can suffer from hallucinations, inconsistent outputs, and sensitivity to prompts. These issues motivate the integration of program analysis and stronger validation mechanisms.

Kevin and Yugopuspito proposed SmartLLM, a smart contract auditing approach using custom generative AI [6]. The study compares conventional tools and general-purpose LLMs with the proposed system. In the reported comparison, Mythril achieved 21.7% accuracy, Slither 35%, ChatGPT-3.5 39.1%, ChatGPT-4 43.3%, while SmartLLM achieved 70.0% accuracy. SmartLLM also reported 100% recall, 62.5% precision, and a 76.9% F1 score in the evaluated setting. The results suggest that domain adaptation and a customized auditing pipeline can improve LLM-based detection. However, the reported evaluation is tied to the selected dataset and auditing configuration, and broader validation is required for general deployment.

Sun et al. proposed GPTScan, which combines GPT with program analysis to detect logic vulnerabilities in smart contracts [7]. Instead of relying only on an LLM or only on static analysis, the approach uses program-analysis information to identify relevant code regions and then applies GPT-based reasoning to analyse potential logic problems. This combination is important because logic vulnerabilities may not match simple syntactic patterns. The study demonstrates that LLM reasoning can complement conventional analysis, but the approach still inherits limitations associated with LLM reliability and the coverage of the underlying program-analysis stage.

Su et al. presented a smart contract vulnerability detection manner based on large language models, focusing specifically on the effect of prompt engineering on vulnerability identification [11]. The study evaluated GPT-3.5 using Solidity vulnerability examples from the SmartBugs Dataset-Wild and considered infinite-loop, integer overflow, integer underflow, and reentrancy cases. The authors compared baseline prompting with Chain-of-Thought (CoT), Plan-and-Solve, and few-shot prompting. Their results showed that prompt design can substantially influence detection performance; for example, in the zero-shot setting, CoT improved infinite-loop detection from 61% to 67% and reentrancy detection from 57.84% to 66.76%. In the few-shot setting, some configurations achieved further gains, including 84% for integer overflow with user-specified Plan-and-Solve instructions. The study demonstrates that LLMs can identify several vulnerability classes without conventional supervised training, but it also highlights limitations such as the 4096-token context of text-davinci-003, knowledge limited to 2021, sensitivity to the selected examples, and the need for better prompt adaptation, retrieval-augmented generation, and fine-tuning.

Du et al. investigated multi-agent debate for improving the factuality and reasoning ability of language models [8]. Multiple language-model agents independently reason about a

problem and then exchange arguments before producing a final answer. The reported experiments on biographies, MMLU, and chess show improvements over single-agent reasoning in the evaluated settings. Although the work is not specifically designed for smart contracts, its central idea is highly relevant to security auditing: independent agents can challenge assumptions, identify missed cases, and reduce the likelihood that one incorrect reasoning path becomes the final result.

Chan et al. introduced ChatEval, a framework designed to improve the reliability of LLM-based evaluation using multi-agent debate [9]. In this approach, several evaluator agents are given different roles or communication strategies and interact with one another before producing a final evaluation. The study shows that using diverse agents and structured communication can lead to more reliable assessment results. This idea can also be applied to smart contract auditing by assigning different agents specific tasks, such as identifying vulnerabilities, verifying supporting evidence, generating counterexamples, and making the final decision. However, using multiple agents also increases computational requirements, and collaboration between agents alone does not guarantee that the final result will always be correct.

Qian et al. introduced ChatDev, a communicative multi-agent framework for software development [10]. The framework organizes specialized agents into software-development roles and uses structured conversational interactions to complete tasks. Its relevance to smart contract auditing lies in its role-based collaboration model. Instead of asking one model to perform all auditing tasks, separate agents can be responsible for code understanding, analysis, testing, documentation, and review. However, ChatDev is a general software-development framework rather than a dedicated smart contract security system, and vague requirements can result in incomplete or simplistic outputs.

Wei et al. proposed LLM-SmartAudit, an advanced smart contract vulnerability detection framework based on LLM-powered multi-agent systems [12]. The approach extends the idea of collaborative reasoning to smart contract auditing by using multiple specialized agents to inspect and reason about vulnerabilities. This design can combine different perspectives and reduce dependence on a single model response. The work represents an important transition from single-model auditing to coordinated security analysis. Nevertheless, multi-agent auditing introduces additional inference cost, coordination complexity, and the need for reliable mechanisms to verify the evidence produced by each agent.

III. COMPARATIVE ANALYSIS

The reviewed studies reveal four major directions in smart contract security: traditional program analysis, supervised machine learning and transformer-based classification, generative LLM auditing, and multi-agent collaborative reasoning. Static analysis provides fast and interpretable rule-based findings, while machine learning models learn vulnerability patterns from labelled datasets. Transformer encoders improve contextual code representation, whereas generative LLMs can reason about code and produce explanations. Multi-agent systems extend this capability by allowing several agents to independently analyse and critique the same contract.

The Table I summarizes the characteristics of the reviewed studies.

From the comparison, static-analysis systems are effective for known vulnerability patterns and are computationally practical, but their coverage is limited by predefined rules. Machine learning and transformer models can learn more complex patterns, yet their performance depends on training data and their explanations may be limited. LLM-based methods provide stronger contextual interpretation and natural-language explanations, but they introduce hallucination and consistency problems. Multi-agent systems provide an additional verification layer by allowing agents to critique one another.

The reviewed studies therefore suggest that a robust auditing platform should not completely replace program analysis with LLMs. Instead, static analysis and learned models can identify candidate locations, while domain-adapted LLM agents can reason about vulnerability semantics. A final verification agent can compare evidence, challenge unsupported claims, and produce an auditable result. Such a hybrid architecture can potentially combine the speed of program analysis with the contextual reasoning ability of LLMs.

IV. RESEARCH GAPS AND FUTURE DIRECTIONS

The reviewed studies show substantial progress, but several gaps remain in building dependable smart contract vulnerability detection systems.

1) Limited coverage of semantic and logic vulnerabilities:

Traditional static analysis performs well on known patterns but may miss vulnerabilities whose correctness depends on contract-specific business logic. GPTScan and related LLM-based methods address this issue, but their reliability depends on the quality of program-analysis inputs and LLM reasoning.

2) False positives and false negatives:

Machine learning and static-analysis approaches can produce incorrect warnings, while LLMs may confidently generate unsupported findings. Future systems should combine independent evidence sources and explicitly require evidence before reporting a vulnerability.

3) Limited Solidity-specific datasets:

Supervised models require labelled vulnerability datasets, and vulnerability categories are often imbalanced. More diverse datasets containing real-world contracts, corrected vulnerabilities, compiler-version variation, and multiple vulnerability labels are required for reliable evaluation.

TABLE I
COMPARISON OF THE REVIEWED SMART CONTRACT VULNERABILITY DETECTION STUDIES

Ref.	Main Approach	Main Technology	Data/Platform	Key Result / Limitation
[1]	Static analysis and predefined vulnerability detectors.	Slither, intermediate representations, program analysis.	Solidity smart contracts.	Fast and practical detection; limited by rule coverage and semantic reasoning.
[2]	Deep-learning vulnerability classification using hybrid attention.	Hybrid Attention Mechanism Model.	Solidity smart contract datasets.	Learns security-relevant representations; dependent on training data.
[3]	Fine-tuned transformer encoder for robust vulnerability detection.	Transformer encoder, pretrained code representations.	Solidity/Ethereum smart contracts.	Strong contextual representation; requires labelled data and limited direct explanations.
[4]	Multi-label and ensemble vulnerability classification.	Ensemble machine learning models.	Solidity vulnerability datasets.	F1 scores reported from 82.0% to 99.9%; distribution shift remains a concern.
[5]	LLM-based smart contract vulnerability detection.	Large Language Models.	Solidity source code.	Provides contextual reasoning; susceptible to hallucination and prompt sensitivity.
[6]	Custom generative-AI auditing pipeline.	LLaMA 3.1, QLoRA, RAG/custom auditing.	Smart contract auditing data and ERC documentation.	SmartLLM reported 70.0% accuracy and 76.9% F1; broader validation needed.
[7]	Logic vulnerability detection combining LLM reasoning and program analysis.	GPT + program analysis.	Solidity smart contracts.	Improves logic-oriented analysis; depends on analysis coverage and LLM reliability.
[8]	Multiple agents debate alternative reasoning paths.	Multi-agent debate with LLMs.	Biography, MMLU, and chess reasoning tasks.	Improved evaluated reasoning performance; not smart-contract-specific.
[9]	Multi-agent evaluation using role diversity and interaction.	ChatEval, LLM evaluators.	LLM evaluation tasks.	More reliable evaluation through debate; increased inference cost.
[10]	Role-based multi-agent software development.	ChatDev, communicative agents.	Software development tasks.	Demonstrates structured agent collaboration; not specialized for security auditing.
[12]	Multi-agent smart contract auditing and vulnerability reasoning.	LLM-powered multi-agent system.	Solidity smart contracts.	Integrates collaborative reasoning for auditing; coordination and validation remain challenging.

4) Hallucination and consistency in LLM auditing:

Generative LLMs can produce plausible but incorrect explanations. Prompt engineering alone is not sufficient for security-critical auditing. Retrieval-augmented information, compiler-aware analysis, static-analysis evidence, test generation, and independent verification agents can help reduce unsupported conclusions.

5) Need for explainable and evidence-based auditing:

A practical auditor should identify the vulnerable location, explain the security property that is violated, provide supporting code evidence, and distinguish confirmed findings from uncertain ones. Multi-agent debate can be extended with explicit evidence checking and confidence estimation.

6) Computational cost of multi-agent systems:

Multi-agent approaches require several model calls and communication steps. This can increase latency and cost when large contracts are audited. Future research should investigate adaptive agent selection, lightweight models, parallel execution, and early stopping when evidence is sufficient.

7) Lack of standardized end-to-end evaluation:

The reviewed studies use different datasets, vulnerability definitions, metrics, and experimental settings. Direct comparison is therefore difficult. Future benchmarks should evaluate precision, recall, F1, false-positive rate, vulnerability localization, explanation correctness, and robustness across unseen contracts.

8) Future hybrid architecture:

A promising direction is a hierarchical auditing system in which static analyzers first generate candidate findings, transformer models classify code patterns, LLM agents perform semantic reasoning, and independent agents act as prosecutor, defender, and judge. The final decision can be accepted only when sufficient code-level evidence supports the finding. This design can improve reliability while preserving the complementary strengths of different techniques.

V. CONCLUSION

This survey reviewed recent approaches for smart contract vulnerability detection, ranging from static analysis and supervised machine learning to transformer models, generative LLMs, and multi-agent auditing systems. Slither demonstrates the practicality of program-analysis-based detection, while attention and transformer models show the benefits of learned code representations. LLM-based approaches such as GPTLENS, SmartLLM, GPTScan, and the prompt-engineering study by Su et al. introduce stronger contextual reasoning, and multi-agent frameworks such as ChatEval, multi-agent debate, ChatDev, and LLM-SmartAudit demonstrate how collaboration and critique can improve reasoning and auditing workflows.

Overall, no single approach completely solves smart contract security auditing. Static analysis offers efficient and evidence-oriented detection, machine learning provides scalable pattern recognition, LLMs provide semantic interpretation, and multi-agent systems provide independent review and debate. The most promising research direction is therefore a hybrid and evidence-driven architecture that combines these capabilities while controlling hallucination, computational cost, and false findings. Such systems can support more reliable, explainable, and practical smart contract security auditing.

ACKNOWLEDGMENT

The authors would like to acknowledge the researchers whose studies were reviewed in this survey and the academic resources used for preparing the literature analysis.

REFERENCES

[1] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *Proc. IEEE/ACM Int. Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019.

[2] H. Wu, H. Dong, Y. He, and Q. Duan, "Smart contract vulnerability detection based on hybrid attention mechanism model," *Applied Sciences*, 2023.

[3] T.-T.-H. Le *et al.*, "Robust vulnerability detection in Solidity-based Ethereum smart contracts using fine-tuned transformer encoder models," *IEEE Access*, 2024.

[4] R. Alnuman, T. Sajid, W. Almobaideen, and Q. Hasan, "Ensemble multi-label machine learning Solidity smart contract vulnerability detection model," *Cluster Computing*, vol. 28, 2025.

[5] S. Hu *et al.*, "Large language model-powered smart contract vulnerability detection: New perspectives," arXiv:2310.01152, 2023.

[6] J. Kevin and P. Yugopuspito, "SmartLLM: Smart contract auditing using custom generative AI," arXiv preprint arXiv:2502.13167, 2025.

[7] Y. Sun *et al.*, "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proc. IEEE/ACM 46th Int. Conf. on Software Engineering (ICSE)*, 2024.

[8] Y. Du *et al.*, "Improving factuality and reasoning in language models through multiagent debate," arXiv preprint arXiv:2305.14325, 2023.

[9] C.-M. Chan *et al.*, "ChatEval: Towards better LLM-based evaluators through multi-agent debate," arXiv preprint arXiv:2308.07201, 2023.

[10] C. Qian *et al.*, "ChatDev: Communicative agents for software development," arXiv preprint arXiv:2307.07924, 2023.

[11] I.-F. Su, S.-M. Wang, Y.-C. Chung, and Y.-H. Tsai, "A smart contract vulnerability detection manner based on large language model," in *Proc. ICKII 2024*, Lecture Notes in Electrical Engineering, vol. 1481, pp. 16–25, 2026, doi: 10.1007/978-981-95-2113-5_2.

[12] Z. Wei *et al.*, "Advanced smart contract vulnerability detection via LLM-powered multi-agent systems," *IEEE Transactions on Software Engineering*, vol. 51, no. 10, pp. 2830–2846, 2025, doi: 10.1109/TSE.2025.3597319.